

The background is a dark blue gradient with a subtle pattern of white dots. On the left side, there are several concentric circular patterns. One large circle has a scale around its perimeter with numbers ranging from 140 to 260 in increments of 10. Other smaller circles and arcs are scattered around, some with arrows indicating a clockwise direction.

POLYMORPHISM

DR. S. VINILA JINNY

POLYMORPHISM

- The word polymorphism is derived from Greek word ***Poly*** which means many and ***morphos*** which means forms.
- Polymorphism can be defined as the ability to use the same name for two or more related but technically different tasks.
- Eg-woman plays role of daughter,sister,wife,mother etc.

TYPES OF POLYMORPHISM

- 1. COMPILE TIME POLYMORPHISM(STATIC)
 - FUNTION OVERLOADING
 - OPERATOR OVERLOADING
- 2. RUNTIME POLYMORPHISM(DYNAMIC)
 - DYNAMIC BINDING(VIRTUAL FUNCTION)

OVERLOADING IN C++

❑ What is overloading

- Overloading means assigning multiple meanings to a function name or operator symbol
- It allows multiple definitions of a function with the same name, but different signatures.

❑ C++ supports

- Function overloading
- Operator overloading

TYPES



C++ FUNCTION OVERLOADING

- Function Overloading is defined as the process of having two or more function with the same name, but different in parameters is known as function overloading in C++.
- In function overloading, the function is redefined by using either different types of arguments or a different number of arguments.
- It is only through these differences compiler can differentiate between the functions.

EXAMPLE

```
#include <iostream>
using namespace std;
class Cal {
    public:
    static int add(int a,int b){
        return a + b;
    }
    static int add(int a, int b, int c)
    {
        return a + b + c;
    }
};
```

```
int main(void) {
    Cal C;
    // class object declaration.
    Cout<<C.add(10, 20)<<endl;
    cout<<C.add(12, 20, 23);
    return 0;
}
```

EXAMPLE

```
#include<iostream>

using namespace std;

int mul(int,int);

float mul(float,int);

int mul(int a,int b)
{
    return a*b;
}

float mul(double x, int y)
{
    return x*y;
}
```

```
int main()
{
    int r1 = mul(6,7);
    float r2 = mul(0.2,3);
    cout << "r1 is : " <<r1<< std::endl;
    cout <<"r2 is : " <<r2<< std::endl;
    return 0;
}
```


OVERLOADING FUNCTION CALL RESOLUTION

- ❑ **Overloaded function call resolution is done by compiler during compilation**
 - **The function signature determines which definition is used**
- ❑ **a Function signature consists of:**
 - **Parameter types and number of parameters supplied to a function**
- ❑ **a Function return type is not part of function signature and is not used in function call resolution**

FUNCTION SELECTION INVOLVES FOLLOWING STEPS.

- **Compiler first tries to find the Exact match in which the type of argument are the same, and uses that func.**
- **If an exact match is not found, the compiler uses the integral promotions to the actual argument such as, char to int, float to double.**
- **When either of them fails, built-in conversions are used (implicit conversion) to the actual arguments and then uses the function whose match is unique. But if there are multiple matches, then the compiler will generate an error message.**

- For ex: **long square(long n)**

long square(double x)

Now a func. call such as **square(10)** will cause an error because int argument can be converted into long also and double also. so it will show ambiguity.

- User defined conversion are followed if all the conversion are failed.
- Function overloading is independent of the return type.

ADVANTAGES OF FUNCTION OVERLOADING

- **Programs execution becomes faster.**
- **Smooth and simple code flow.**
- **Code maintenance becomes easier.**
- **Improves code readability.**
- **Saves memory space.**
- **Code reusability achieved.**
- **It brings flexibility to the code.**
- **It can perform different operations, and hence it eliminates the use of different function names for the same kind of operations.**

DISADVANTAGES OF FUNCTION OVERLOADING

- Functions with different return types cannot be overloaded as they can have the same parameter definition.
- It cannot overload functions with the same name and parameter if any one of them is a static member function declaration.

OVERLOADING AMBIGUITY

- **The situation in which the compiler is unable to decide the appropriate overloaded function is called overloading ambiguity. In that case, the compiler won't run the program.**

OVERLOADING AMBIGUITY OCCURS DURING

- Type conversions
- Functions with default arguments
- Functions with pass by reference

AMBIGUITY ON TYPE CONVERSION

```
#include<iostream>

using namespace std;

void function(float) {
    cout << "Data Type: float\n";
}

void function(int) {
    cout << "Data Type: int\n";
}
```

```
int main() {
    function(1.0);
    function(1);
    return 0;
}
```


AMBIGUITY ON FUNCTION WITH DEFAULT ARGUMENTS

```
#include<iostream>
```

```
using namespace std;
```

```
int add(int a) {
```

```
    int b = 10;
```

```
    return a + b;
```

```
}
```

```
int add(int a, int b = 10) {
```

```
    return a + b;
```

```
}
```

```
int main() {
```

```
    int a = 5;
```

```
    cout << "a + b = " << add(a) << endl;
```

```
    return 0;
```

```
}
```

AMBIGUITY ON FUNCTION WITH PASS BY REFERENCE

```
#include<iostream>
using namespace std;
void display(int a) {
    cout << "a = "<<a<<endl;
}
void display(int &a) {
    cout << "a = "<<a<<endl;
}
```

```
int main() {
    int a = 5;
    display(a);
    return 0;
}
```

CAN THE MAIN() FUNCTION BE OVERLOADED IN C++?

YES, THE MAIN() FUNCTION CAN BE OVERLOADED IN C++.

```
#include <iostream>
using namespace std;
```

```
// create a Main class and declare main() as member
function
```

```
class Main {
public:
```

```
    int main(int a) {
        cout<<"a = "<<a<<endl;
        return 0;
    }
```

```
int main(int a ,int b) {
    cout<<"a = "<<a<<" b = "<<b<<endl;
    return 0;
}
};
```

```
int main() {
    Main object;
    object.main(5);
    object.main(5,10);
    return 0;
}
```